# WPI ECE 2049 STM32 Adafruit TFT 1.8" Shield

Generated by Doxygen 1.16.1

**1 Directory Hierarchy** 1
1.1 Directories . . . 1

**2 File Index** 3
2.1 File List . . . 3

**3 Directory Documentation** 5
3.1 Core Directory Reference . . . 5
3.2 Core/Inc/graphics Directory Reference . . . 5
3.3 Core/Inc Directory Reference . . . 5

**4 File Documentation** 7
4.1 Core/Inc/graphics/Adafruit_ST7735_API.h File Reference . . . 7
4.1.1 Typedef Documentation . . . 10
4.1.1.1 ST7735_Handle . . . 10
4.1.2 Function Documentation . . . 10
4.1.2.1 Display_color565() . . . 10
4.1.2.2 Display_cp437() . . . 10
4.1.2.3 Display_create() . . . 11
4.1.2.4 Display_drawBitmap() . . . 11
4.1.2.5 Display_drawBitmapBg() . . . 11
4.1.2.6 Display_drawChar() . . . 12
4.1.2.7 Display_drawCircle() . . . 12
4.1.2.8 Display_drawCircleHelper() . . . 13
4.1.2.9 Display_drawEllipse() . . . 13
4.1.2.10 Display_drawFastHLine() . . . 14
4.1.2.11 Display_drawFastVLine() . . . 14
4.1.2.12 Display_drawGrayscaleBitmap() . . . 14
4.1.2.13 Display_drawGrayscaleBitmapMask() . . . 15
4.1.2.14 Display_drawLine() . . . 15
4.1.2.15 Display_drawPixel() . . . 16
4.1.2.16 Display_drawRect() . . . 16
4.1.2.17 Display_drawRGBBitmap() . . . 16
4.1.2.18 Display_drawRGBBitmapMask() . . . 17
4.1.2.19 Display_drawRoundRect() . . . 17
4.1.2.20 Display_drawTriangle() . . . 18
4.1.2.21 Display_drawXBitmap() . . . 18
4.1.2.22 Display_drawXBitmapBg() . . . 19
4.1.2.23 Display_enableDisplay() . . . 19
4.1.2.24 Display_enableSleep() . . . 20
4.1.2.25 Display_enableTearing() . . . 20
4.1.2.26 Display_fillCircle() . . . 20
4.1.2.27 Display_fillCircleHelper() . . . 20

4.1.2.28 Display_fillEllipse() . . . 21
4.1.2.29 Display_fillRect() . . . 21
4.1.2.30 Display_fillRoundRect() . . . 22
4.1.2.31 Display_fillScreen() . . . 22
4.1.2.32 Display_fillTriangle() . . . 22
4.1.2.33 Display_getCursorX() . . . 23
4.1.2.34 Display_getCursorY() . . . 23
4.1.2.35 Display_getRotation() . . . 23
4.1.2.36 Display_getTextBounds() . . . 24
4.1.2.37 Display_height() . . . 24
4.1.2.38 Display_init() . . . 24
4.1.2.39 Display_invertDisplay() . . . 25
4.1.2.40 Display_print() . . . 25
4.1.2.41 Display_println() . . . 25
4.1.2.42 Display_setAddrWindow() . . . 25
4.1.2.43 Display_setCursor() . . . 26
4.1.2.44 Display_setFont() . . . 26
4.1.2.45 Display_setRotation() . . . 26
4.1.2.46 Display_setTextColor() . . . 27
4.1.2.47 Display_setTextColorWithBg() . . . 27
4.1.2.48 Display_setTextSize() . . . 27
4.1.2.49 Display_setTextWrap() . . . 27
4.1.2.50 Display_width() . . . 28
4.2 Adafruit_ST7735_API.h . . . 28
4.3 Core/Inc/graphics/Adafruit_TFTShield18_API.h File Reference . . . 30
4.3.1 Typedef Documentation . . . 30
4.3.1.1 TFTShield18_Handle . . . 30
4.3.2 Function Documentation . . . 31
4.3.2.1 TFTShield18_begin() . . . 31
4.3.2.2 TFTShield18_create() . . . 31
4.3.2.3 TFTShield18_readButtons() . . . 31
4.3.2.4 TFTShield18_setBacklight() . . . 31
4.3.2.5 TFTShield18_setBacklightFreq() . . . 32
4.3.2.6 TFTShield18_tftReset() . . . 32
4.4 Adafruit_TFTShield18_API.h . . . 32

**Index** **33**

# Chapter 1

# Directory Hierarchy

## 1.1 Directories


Core . . . . . 5
- Inc . . . . . 5
  - graphics . . . . . 5
    - Adafruit_ST7735_API.h . . . . . 7
    - Adafruit_TFTShield18_API.h . . . . . 30

graphics . . . . . 5
- Adafruit_ST7735_API.h . . . . . 7
- Adafruit_TFTShield18_API.h . . . . . 30

Inc . . . . . 5
- graphics . . . . . 5
  - Adafruit_ST7735_API.h . . . . . 7
  - Adafruit_TFTShield18_API.h . . . . . 30

# Chapter 2

# File Index

## 2.1 File List

Here is a list of all files with brief descriptions:

Core/Inc/graphics/Adafruit_ST7735_API.h . . . . . 7
Core/Inc/graphics/Adafruit_TFTShield18_API.h . . . . . 30

# Chapter 3

# Directory Documentation

## 3.1 Core Directory Reference

### Directories

- directory Inc

## 3.2 Core/Inc/graphics Directory Reference

### Files

- file Adafruit_ST7735_API.h
- file Adafruit_TFTShield18_API.h

## 3.3 Core/Inc Directory Reference

### Directories

- directory graphics

# Chapter 4

# File Documentation

## 4.1 Core/Inc/graphics/Adafruit_ST7735_API.h File Reference

```
#include "stdint.h"
#include "stm32h5xx_hal.h"
#include "stdbool.h"
#include "Adafruit_ST7735_Macros.h"
#include "Adafruit_ST77xx_Macros.h"
#include "gfxfont.h"
#include "stm32h5xx_hal_spi.h"
```

### Typedefs

- typedef struct ST7735_Handle ST7735_Handle

  *Internal handle used to store the instance of an Adafruit_ST7735 class for use in the C API functions. This struct is not intended to be used outside of this API and should be treated as an opaque handle. Provides a C compatible structure to store the C++ class instance for the ST7735 display.*

### Functions

- void Display_create (SPI_HandleTypeDef ∗spi_handle, int16_t cs, GPIO_TypeDef ∗cs_port, int16_t dc, GPIO_TypeDef ∗dc_port)

  *Function used to create the instance for the display's graphics.*
- void Display_init (uint8_t options)

  *Function used to initialize the display using the given options.*
- void Display_setRotation (uint8_t m)

  *Function to change rotation of display.*
- void Display_setAddrWindow (uint16_t x, uint16_t y, uint16_t w, uint16_t h)

  *Sets the address windows that the display expects to receive data in from another function call. Typically not used as singleton.*
- void Display_enableDisplay (bool enable)

  *Turns the display on or off.*
- void Display_enableTearing (bool enable)

  *Enables a display tearing correction pin. Unused on the TFT 1.8" display.*
- void Display_enableSleep (bool enable)

*Enables whether the display should be able to enter sleep mode when given a command. Not typically used.*

- void Display_drawPixel (int16_t x, int16_t y, uint16_t color)

  *Draws a single pixel with a color to the display. Singleton function that automatically calls setAddrWindow.*
- void Display_fillRect (int16_t x, int16_t y, int16_t w, int16_t h, uint16_t color)

  *Fills a rectangle on the display with a color. Singleton function that automatically calls setAddrWindow.*
- void Display_drawFastHLine (int16_t x, int16_t y, int16_t w, uint16_t color)

  *Optimized horizontal line drawing function. Singleton function that automatically calls setAddrWindow.*
- void Display_drawFastVLine (int16_t x, int16_t y, int16_t h, uint16_t color)

  *Optimized vertical line drawing function. Singleton function that automatically calls setAddrWindow.*
- void Display_invertDisplay (bool i)

  *Function to invert the display colors. Singleton function that automatically calls setAddrWindow.*
- uint16_t Display_color565 (uint8_t r, uint8_t g, uint8_t b)

  *Converts 8-bit R, G, and B values into a single 16-bit value in 565 RGB format.*
- void Display_drawRGBBitmap (int16_t x, int16_t y, uint16_t ∗pcolors, int16_t w, int16_t h)

  *Function to draw an RGB bitmap to the display. Singleton function that automatically calls setAddrWindow.*
- void Display_drawRGBBitmapMask (int16_t x, int16_t y, uint16_t ∗pcolors, int16_t w, int16_t h, uint8_t ∗mask)

  *Function to draw an RGB bitmap to the display with an optional mask. Singleton function that automatically calls setAddrWindow.*
- void Display_fillScreen (uint16_t color)

  *Function to fill the entire screen with a single color. Singleton function that automatically calls setAddrWindow.*
- void Display_drawLine (int16_t x0, int16_t y0, int16_t x1, int16_t y1, uint16_t color)

  *Function to draw a line between two points. Singleton function that automatically calls setAddrWindow.*
- void Display_drawRect (int16_t x, int16_t y, int16_t w, int16_t h, uint16_t color)

  *Function to draw a non-filled rectangle. Singleton function that automatically calls setAddrWindow.*
- void Display_drawCircle (int16_t x0, int16_t y0, int16_t r, uint16_t color)

  *Function to draw a non-filled circle. Singleton function that automatically calls setAddrWindow.*
- void Display_drawCircleHelper (int16_t x0, int16_t y0, int16_t r, uint8_t cornername, uint16_t color)

  *Function to draw a circle that you can choose the corner to draw. Singleton function that automatically calls set↩ AddrWindow.*
- void Display_fillCircle (int16_t x0, int16_t y0, int16_t r, uint16_t color)

  *Function to draw a filled circle. Singleton function that automatically calls setAddrWindow.*
- void Display_fillCircleHelper (int16_t x0, int16_t y0, int16_t r, uint8_t cornername, int16_t delta, uint16_t color)

  *Function to draw a filled circle that you can choose the corner to draw. Singleton function that automatically calls setAddrWindow.*
- void Display_drawEllipse (int16_t x0, int16_t y0, int16_t rw, int16_t rh, uint16_t color)

  *Function to draw a non-filled ellipse. Singleton function that automatically calls setAddrWindow.*
- void Display_fillEllipse (int16_t x0, int16_t y0, int16_t rw, int16_t rh, uint16_t color)

  *Function to draw a filled ellipse. Singleton function that automatically calls setAddrWindow.*
- void Display_drawTriangle (int16_t x0, int16_t y0, int16_t x1, int16_t y1, int16_t x2, int16_t y2, uint16_t color)

  *Function to draw a non-filled triangle. Singleton function that automatically calls setAddrWindow.*
- void Display_fillTriangle (int16_t x0, int16_t y0, int16_t x1, int16_t y1, int16_t x2, int16_t y2, uint16_t color)

  *Function to draw a filled triangle. Singleton function that automatically calls setAddrWindow.*
- void Display_drawRoundRect (int16_t x0, int16_t y0, int16_t w, int16_t h, int16_t radius, uint16_t color)

  *Draws a rounded non-filled rectangle. Singleton function that automatically calls setAddrWindow.*
- void Display_fillRoundRect (int16_t x0, int16_t y0, int16_t w, int16_t h, int16_t radius, uint16_t color)

  *Draws a filled rounded rectangle. Singleton function that automatically calls setAddrWindow.*
- void Display_drawBitmap (int16_t x, int16_t y, uint8_t ∗bitmap, int16_t w, int16_t h, uint16_t color)

  *Draws a monochrome bitmap to the display. Singleton function that automatically calls setAddrWindow.*
- void Display_drawBitmapBg (int16_t x, int16_t y, uint8_t ∗bitmap, int16_t w, int16_t h, uint16_t color, uint16_t bg)

  *Draws a monochrome bitmap with a background color to the display. Singleton function that automatically calls setAddrWindow.*

- void Display_drawXBitmap (int16_t x, int16_t y, uint8_t *bitmap, int16_t w, int16_t h, uint16_t color)

  *Draws an XBM format bitmap exported from GIMP to the display. Singleton function that automatically calls setAddrWindow.*
- void Display_drawXBitmapBg (int16_t x, int16_t y, uint8_t *bitmap, int16_t w, int16_t h, uint16_t color, uint16_t bg)

  *Draws an XBM format bitmap exported from GIMP to the display with a background color. Singleton function that automatically calls setAddrWindow.*
- void Display_drawGrayscaleBitmap (int16_t x, int16_t y, uint8_t *bitmap, int16_t w, int16_t h)

  *Draws a grayscale bitmap to the display. Singleton function that automatically calls setAddrWindow.*
- void Display_drawGrayscaleBitmapMask (int16_t x, int16_t y, uint8_t *bitmap, int16_t w, int16_t h, uint8_t *mask)

  *Draws a grayscale bitmap with a mask to the display. Singleton function that automatically calls setAddrWindow.*
- void Display_drawChar (int16_t x, int16_t y, unsigned char c, uint16_t color, uint16_t bg, uint8_t size_x, uint8_t size_y)

  *Draws a character to the display. Singleton function that automatically calls setAddrWindow.*
- void Display_getTextBounds (const char *string, int16_t x, int16_t y, int16_t *x1, int16_t *y1, uint16_t *w, uint16_t *h)

  *Gets the bounding box of a string of text. Used to determine how much space a string of text will take up on the display. Singleton function that does not call setAddrWindow.*
- void Display_setTextSize (uint8_t sx, uint8_t sy)

  *Function to set the text size for the display.*
- void Display_setFont (const GFXfont *f)

  *Function to set the font for the display.*
- void Display_setCursor (int16_t x, int16_t y)

  *Sets the cursor position for the display. The cursor is used as the starting point for drawing text. Singleton function that does not call setAddrWindow.*
- void Display_setTextColor (uint16_t c)

  *Sets the color used to draw text on the display.*
- void Display_setTextColorWithBg (uint16_t c, uint16_t bg)

  *Sets the text color and background color behind the characters.*
- void Display_setTextWrap (bool w)

  *Sets whether text should wrap when it reaches the edge of the display.*
- void Display_cp437 (bool x)

  *Sets whether to use the CP437 character set. If false, will use standard ASCII. If true, will use CP437 which has some additional characters in the 128-255 range. Singleton function that does not call setAddrWindow.*
- int16_t Display_width ()

  *Get the width of the initialized display. Adjusts based on the current rotation of the display.*
- int16_t Display_height ()

  *Get the height of the initialized display. Adjusts based on the current rotation of the display.*
- uint8_t Display_getRotation ()

  *Get the current rotation of the display.*
- int16_t Display_getCursorX ()

  *Gets the current x coord of the display's cursor.*
- int16_t Display_getCursorY ()

  *Gets the current y coord of the display's cursor.*
- void Display_print (const char *str)

  *Print a string of text to the display.*
- void Display_println (const char *str)

  *Print a string of text to the display followed by a newline.*

### 4.1.1 Typedef Documentation

#### 4.1.1.1 ST7735_Handle

```
typedef struct ST7735_Handle ST7735_Handle
```

Internal handle used to store the instance of an Adafruit_ST7735 class for use in the C API functions. This struct is not intended to be used outside of this API and should be treated as an opaque handle. Provides a C compatible structure to store the C++ class instance for the ST7735 display.

### 4.1.2 Function Documentation

#### 4.1.2.1 Display_color565()

```
uint16_t Display_color565 (
            uint8_t r,
            uint8_t g,
            uint8_t b)
```

Converts 8-bit R, G, and B values into a single 16-bit value in 565 RGB format.

**Parameters**

| | |
|---|---|
| *r* | 8-bit red value |
| *g* | 8-bit green value |
| *b* | 8-bit blue value |

**Returns**

uint16_t 16-bit color value in 565 RGB format

#### 4.1.2.2 Display_cp437()

```
void Display_cp437 (
            bool x)
```

Sets whether to use the CP437 character set. If false, will use standard ASCII. If true, will use CP437 which has some additional characters in the 128-255 range. Singleton function that does not call setAddrWindow.

**Parameters**

| | |
|---|---|
| *x* | true to enable CP437 character set, false to use standard ASCII |

#### 4.1.2.3 Display_create()

```
void Display_create (
            SPI_HandleTypeDef * spi_handle,
            int16_t cs,
            GPIO_TypeDef * cs_port,
            int16_t dc,
            GPIO_TypeDef * dc_port)
```

Function used to create the instance for the display's graphics.

**Parameters**

| | |
|---|---|
| *spi_handle* | The handle for the STM32 hardware spi instance that has been set up for the TFT display |
| *cs* | The pin number for the TFT display's CS pin. Will come from a macro with the pin label generated by STM32CubeMX |
| *cs_port* | The STM32 GPIO port for the TFT display's CS pin. Predefined macro for the GPIO port associated with the CS pin |
| *dc* | The pin number for the TFT display's DC pin. Will come from a macro with the pin label generated by STM32CubeMX |
| *dc_port* | The STM32 GPIO port for the TFT display's DC pin. Predefined macro for the GPIO port associated with the DC pin |

#### 4.1.2.4 Display_drawBitmap()

```
void Display_drawBitmap (
            int16_t x,
            int16_t y,
            uint8_t * bitmap,
            int16_t w,
            int16_t h,
            uint16_t color)
```

Draws a monochrome bitmap to the display. Singleton function that automatically calls setAddrWindow.

**Parameters**

| | |
|---|---|
| *x* | Top-left corner x coord of the bitmap |
| *y* | Top-left corner y coord of the bitmap |
| *bitmap* | Pointer to the bitmap data |
| *w* | Width of the bitmap |
| *h* | Height of the bitmap |
| *color* | Color to draw the bitmap |

#### 4.1.2.5 Display_drawBitmapBg()

```
void Display_drawBitmapBg (
            int16_t x,
            int16_t y,
```

```
            uint8_t * bitmap,
            int16_t w,
            int16_t h,
            uint16_t color,
            uint16_t bg)
```

Draws a monochrome bitmap with a background color to the display. Singleton function that automatically calls setAddrWindow.

**Parameters**

| | |
|---|---|
| *x* | Top-left corner x coord of the bitmap |
| *y* | Top-left corner y coord of the bitmap |
| *bitmap* | Pointer to the bitmap data |
| *w* | Width of the bitmap |
| *h* | Height of the bitmap |
| *color* | Color to draw the bitmap |
| *bg* | Background color of the bitmap |

#### 4.1.2.6 Display_drawChar()

```
void Display_drawChar (
            int16_t x,
            int16_t y,
            unsigned char c,
            uint16_t color,
            uint16_t bg,
            uint8_t size_x,
            uint8_t size_y)
```

Draws a character to the display. Singleton function that automatically calls setAddrWindow.

**Parameters**

| | |
|---|---|
| *x* | Top-left corner x coord of the character |
| *y* | Top-left corner y coord of the character |
| *c* | Character to draw |
| *color* | Color of the character |
| *bg* | Background color of the character |
| *size↩ _x* | Horizontal scaling factor |
| *size↩ _y* | Vertical scaling factor |

#### 4.1.2.7 Display_drawCircle()

```
void Display_drawCircle (
            int16_t x0,
```

```
            int16_t y0,
            int16_t r,
            uint16_t color)
```

Function to draw a non-filled circle. Singleton function that automatically calls setAddrWindow.

**Parameters**

| | |
|---|---|
| *x0* | Center x coord of the circle |
| *y0* | Center y coord of the circle |
| *r* | Radius of the circle |
| *color* | Color of the circle |

#### 4.1.2.8 Display_drawCircleHelper()

```
void Display_drawCircleHelper (
            int16_t x0,
            int16_t y0,
            int16_t r,
            uint8_t cornername,
            uint16_t color)
```

Function to draw a circle that you can choose the corner to draw. Singleton function that automatically calls set↩ AddrWindow.

**Parameters**

| | |
|---|---|
| *x0* | Center x coord of the circle |
| *y0* | Center y coord of the circle |
| *r* | Radius of the circle |
| *cornername* | Corner of the circle to draw. Mask bit #1, #2, #4, and #8 to indicate which quarters of the circle should be drawn |
| *color* | Color of the circle |

#### 4.1.2.9 Display_drawEllipse()

```
void Display_drawEllipse (
            int16_t x0,
            int16_t y0,
            int16_t rw,
            int16_t rh,
            uint16_t color)
```

Function to draw a non-filled ellipse. Singleton function that automatically calls setAddrWindow.

**Parameters**

| | |
|---|---|
| *x0* | Center x coord of the ellipse |

| | |
|---|---|
| *y0* | Center y coord of the ellipse |
| *rw* | Radius along the x-axis |
| *rh* | Radius along the y-axis |
| *color* | Color of the ellipse |

#### 4.1.2.10 Display_drawFastHLine()

```
void Display_drawFastHLine (
            int16_t x,
            int16_t y,
            int16_t w,
            uint16_t color)
```

Optimized horizontal line drawing function. Singleton function that automatically calls setAddrWindow.

**Parameters**

| | |
|---|---|
| *x* | Starting x coord for the line |
| *y* | The y coord to draw the line on |
| *w* | The width of the line |
| *color* | Color to draw the line in |

#### 4.1.2.11 Display_drawFastVLine()

```
void Display_drawFastVLine (
            int16_t x,
            int16_t y,
            int16_t h,
            uint16_t color)
```

Optimized vertical line drawing function. Singleton function that automatically calls setAddrWindow.

**Parameters**

| | |
|---|---|
| *x* | X coord to draw the line on |
| *y* | Starting y coord for the line |
| *h* | Height of the line |
| *color* | Color to draw the line in |

#### 4.1.2.12 Display_drawGrayscaleBitmap()

```
void Display_drawGrayscaleBitmap (
            int16_t x,
            int16_t y,
            uint8_t * bitmap,
```

```
            int16_t w,
            int16_t h)
```

Draws a grayscale bitmap to the display. Singleton function that automatically calls setAddrWindow.

**Parameters**

| | |
|---|---|
| *x* | Top-left corner x coord of the bitmap |
| *y* | Top-left corner y coord of the bitmap |
| *bitmap* | Pointer to the bitmap data |
| *w* | Width of the bitmap |
| *h* | Height of the bitmap |

#### 4.1.2.13 Display_drawGrayscaleBitmapMask()

```
void Display_drawGrayscaleBitmapMask (
            int16_t x,
            int16_t y,
            uint8_t * bitmap,
            int16_t w,
            int16_t h,
            uint8_t * mask)
```

Draws a grayscale bitmap with a mask to the display. Singleton function that automatically calls setAddrWindow.

**Parameters**

| | |
|---|---|
| *x* | Top-left corner x coord of the bitmap |
| *y* | Top-left corner y coord of the bitmap |
| *bitmap* | Pointer to the bitmap data |
| *w* | Width of the bitmap |
| *h* | Height of the bitmap |
| *mask* | Pointer to the mask data (1=bit mask where 1 is set and 0 is clear) |

#### 4.1.2.14 Display_drawLine()

```
void Display_drawLine (
            int16_t x0,
            int16_t y0,
            int16_t x1,
            int16_t y1,
            uint16_t color)
```

Function to draw a line between two points. Singleton function that automatically calls setAddrWindow.

**Parameters**

| | |
|---|---|
| *x0* | Starting x coord of the line |

| | |
|---|---|
| *y0* | Starting y coord of the line |
| *x1* | Ending x coord of the line |
| *y1* | Ending y coord of the line |
| *color* | Color of the line |

#### 4.1.2.15 Display_drawPixel()

```
void Display_drawPixel (
            int16_t x,
            int16_t y,
            uint16_t color)
```

Draws a single pixel with a color to the display. Singleton function that automatically calls setAddrWindow.

**Parameters**

| | |
|---|---|
| *x* | X coord of the pixel |
| *y* | Y coord of the pixel |
| *color* | Color to set the pixel |

#### 4.1.2.16 Display_drawRect()

```
void Display_drawRect (
            int16_t x,
            int16_t y,
            int16_t w,
            int16_t h,
            uint16_t color)
```

Function to draw a non-filled rectangle. Singleton function that automatically calls setAddrWindow.

**Parameters**

| | |
|---|---|
| *x* | Starting x coord of the rectangle |
| *y* | Starting y coord of the rectangle |
| *w* | Width of the rectangle |
| *h* | Height of the rectangle |
| *color* | Color of the rectangle |

#### 4.1.2.17 Display_drawRGBBitmap()

```
void Display_drawRGBBitmap (
            int16_t x,
            int16_t y,
            uint16_t * pcolors,
```

```
            int16_t w,
            int16_t h)
```

Function to draw an RGB bitmap to the display. Singleton function that automatically calls setAddrWindow.

**Parameters**

| | |
|---|---|
| *x* | Starting x coord to draw the bitmap at the top left corner |
| *y* | Starting y coord to draw the bitmap at the top left corner |
| *pcolors* | Pointer to the bitmap data in an array of 16-bit color values in 565 RGB format |
| *w* | Width of the bitmap |
| *h* | Height of the bitmap |

#### 4.1.2.18 Display_drawRGBBitmapMask()

```
void Display_drawRGBBitmapMask (
            int16_t x,
            int16_t y,
            uint16_t * pcolors,
            int16_t w,
            int16_t h,
            uint8_t * mask)
```

Function to draw an RGB bitmap to the display with an optional mask. Singleton function that automatically calls setAddrWindow.

**Parameters**

| | |
|---|---|
| *x* | Starting x coord to draw the bitmap at the top left corner |
| *y* | Starting y coord to draw the bitmap at the top left corner |
| *pcolors* | Pointer to the bitmap data in an array of 16-bit color values in 565 RGB format |
| *w* | Width of the bitmap |
| *h* | Height of the bitmap |
| *mask* | Pointer to the mask data in an array of 8-bit values |

#### 4.1.2.19 Display_drawRoundRect()

```
void Display_drawRoundRect (
            int16_t x0,
            int16_t y0,
            int16_t w,
            int16_t h,
            int16_t radius,
            uint16_t color)
```

Draws a rounded non-filled rectangle. Singleton function that automatically calls setAddrWindow.

**Parameters**

| | |
|---|---|
| *x0* | Top-left corner x coord of the rectangle |
| *y0* | Top-left corner y coord of the rectangle |
| *w* | Width of the rectangle |
| *h* | Height of the rectangle |
| *radius* | Radius of the rounded corners |
| *color* | Color of the rectangle |

#### 4.1.2.20 Display_drawTriangle()

```
void Display_drawTriangle (
            int16_t x0,
            int16_t y0,
            int16_t x1,
            int16_t y1,
            int16_t x2,
            int16_t y2,
            uint16_t color)
```

Function to draw a non-filled triangle. Singleton function that automatically calls setAddrWindow.

**Parameters**

| | |
|---|---|
| *x0* | First vertex x coord of the triangle |
| *y0* | First vertex y coord of the triangle |
| *x1* | Second vertex x coord of the triangle |
| *y1* | Second vertex y coord of the triangle |
| *x2* | Third vertex x coord of the triangle |
| *y2* | Third vertex y coord of the triangle |
| *color* | Color of the triangle |

#### 4.1.2.21 Display_drawXBitmap()

```
void Display_drawXBitmap (
            int16_t x,
            int16_t y,
            uint8_t * bitmap,
            int16_t w,
            int16_t h,
            uint16_t color)
```

Draws an XBM format bitmap exported from GIMP to the display. Singleton function that automatically calls set↩ AddrWindow.

**Parameters**

| | |
|---|---|
| *x* | Top-left corner x coord of the bitmap |
| *y* | Top-left corner y coord of the bitmap |
| *bitmap* | Pointer to the bitmap data |
| *w* | Width of the bitmap |
| *h* | Height of the bitmap |
| *color* | Color to draw the bitmap |

#### 4.1.2.22 Display_drawXBitmapBg()

```
void Display_drawXBitmapBg (
            int16_t x,
            int16_t y,
            uint8_t * bitmap,
            int16_t w,
            int16_t h,
            uint16_t color,
            uint16_t bg)
```

Draws an XBM format bitmap exported from GIMP to the display with a background color. Singleton function that automatically calls setAddrWindow.

**Parameters**

| | |
|---|---|
| *x* | Top-left corner x coord of the bitmap |
| *y* | Top-left corner y coord of the bitmap |
| *bitmap* | Pointer to the bitmap data |
| *w* | Width of the bitmap |
| *h* | Height of the bitmap |
| *color* | Color to draw the bitmap |

#### 4.1.2.23 Display_enableDisplay()

```
void Display_enableDisplay (
            bool enable)
```

Turns the display on or off.

**Parameters**

| | |
|---|---|
| *enable* | True turns the display on, false turns the display off |

#### 4.1.2.24 Display_enableSleep()

```
void Display_enableSleep (
            bool enable)
```

Enables whether the display should be able to enter sleep mode when given a command. Not typically used.

**Parameters**

| | |
|---|---|
| *enable* | True enables sleeping, flase disables sleeping |

#### 4.1.2.25 Display_enableTearing()

```
void Display_enableTearing (
            bool enable)
```

Enables a display tearing correction pin. Unused on the TFT 1.8" display.

**Parameters**

| | |
|---|---|
| *enable* | True turns enables tearing correct, false disables tearing correction |

#### 4.1.2.26 Display_fillCircle()

```
void Display_fillCircle (
            int16_t x0,
            int16_t y0,
            int16_t r,
            uint16_t color)
```

Function to draw a filled circle. Singleton function that automatically calls setAddrWindow.

**Parameters**

| | |
|---|---|
| *x0* | Center x coord of the circle |
| *y0* | Center y coord of the circle |
| *r* | Radius of the circle |
| *color* | Color of the circle |

#### 4.1.2.27 Display_fillCircleHelper()

```
void Display_fillCircleHelper (
            int16_t x0,
            int16_t y0,
            int16_t r,
            uint8_t cornername,
            int16_t delta,
            uint16_t color)
```

Function to draw a filled circle that you can choose the corner to draw. Singleton function that automatically calls setAddrWindow.

**Parameters**

| | |
|---|---|
| *x0* | Center x coord of the circle |
| *y0* | Center y coord of the circle |
| *r* | Radius of the circle |
| *cornername* | Corner of the circle to draw. Mask bit #1, #2, #4, and #8 to indicate which quarters of the circle should be drawn |
| *delta* | Offset from the circle's radius |
| *color* | Color of the circle |

#### 4.1.2.28 Display_fillEllipse()

```
void Display_fillEllipse (
            int16_t x0,
            int16_t y0,
            int16_t rw,
            int16_t rh,
            uint16_t color)
```

Function to draw a filled ellipse. Singleton function that automatically calls setAddrWindow.

**Parameters**

| | |
|---|---|
| *x0* | Center x coord of the ellipse |
| *y0* | Center y coord of the ellipse |
| *rw* | Radius along the x-axis |
| *rh* | Radius along the y-axis |
| *color* | Color of the ellipse |

#### 4.1.2.29 Display_fillRect()

```
void Display_fillRect (
            int16_t x,
            int16_t y,
            int16_t w,
            int16_t h,
            uint16_t color)
```

Fills a rectangle on the display with a color. Singleton function that automatically calls setAddrWindow.

**Parameters**

| | |
|---|---|
| *x* | X coord of the top left corner of the rectangle |
| *y* | Y coord of the top left corner of the rectangle |
| *w* | Width of the rectangle |

| | |
|---|---|
| *h* | Height of the rectangle |
| *color* | Color to fill the rectangle with. In the (565) bit RGB format |

#### 4.1.2.30 Display_fillRoundRect()

```
void Display_fillRoundRect (
            int16_t x0,
            int16_t y0,
            int16_t w,
            int16_t h,
            int16_t radius,
            uint16_t color)
```

Draws a filled rounded rectangle. Singleton function that automatically calls setAddrWindow.

**Parameters**

| | |
|---|---|
| *x0* | Top-left corner x coord of the rectangle |
| *y0* | Top-left corner y coord of the rectangle |
| *w* | Width of the rectangle |
| *h* | Height of the rectangle |
| *radius* | Radius of the rounded corners |
| *color* | Color of the rectangle |

#### 4.1.2.31 Display_fillScreen()

```
void Display_fillScreen (
            uint16_t color)
```

Function to fill the entire screen with a single color. Singleton function that automatically calls setAddrWindow.

**Parameters**

| | |
|---|---|
| *color* | Color to fill the screen with |

#### 4.1.2.32 Display_fillTriangle()

```
void Display_fillTriangle (
            int16_t x0,
            int16_t y0,
            int16_t x1,
            int16_t y1,
            int16_t x2,
            int16_t y2,
            uint16_t color)
```

Function to draw a filled triangle. Singleton function that automatically calls setAddrWindow.

**Parameters**

| | |
|---|---|
| *x0* | First vertex x coord of the triangle |
| *y0* | First vertex y coord of the triangle |
| *x1* | Second vertex x coord of the triangle |
| *y1* | Second vertex y coord of the triangle |
| *x2* | Third vertex x coord of the triangle |
| *y2* | Third vertex y coord of the triangle |
| *color* | Color of the triangle |

#### 4.1.2.33 Display_getCursorX()

```
int16_t Display_getCursorX ()
```

Gets the current x coord of the display's cursor.

**Returns**

int16_t the current x coord of the display's cursor

#### 4.1.2.34 Display_getCursorY()

```
int16_t Display_getCursorY ()
```

Gets the current y coord of the display's cursor.

**Returns**

int16_t the current y coord of the display's cursor

#### 4.1.2.35 Display_getRotation()

```
uint8_t Display_getRotation ()
```

Get the current rotation of the display.

**Returns**

uint8_t rotation of display (0-3 corresponding to 0, 90, 180, and 270 degrees)

#### 4.1.2.36 Display_getTextBounds()

```
void Display_getTextBounds (
            const char * string,
            int16_t x,
            int16_t y,
            int16_t * x1,
            int16_t * y1,
            uint16_t * w,
            uint16_t * h)
```

Gets the bounding box of a string of text. Used to determine how much space a string of text will take up on the display. Singleton function that does not call setAddrWindow.

**Parameters**

| | |
|---|---|
| *string* | Pointer to the string of text |
| *x* | Top-left corner x coord of the text |
| *y* | Top-left corner y coord of the text |
| *x1* | Pointer to store the top-left corner x coord of the bounding box |
| *y1* | Pointer to store the top-left corner y coord of the bounding box |
| *w* | Pointer to store the width of the bounding box |
| *h* | Pointer to store the height of the bounding box |

#### 4.1.2.37 Display_height()

```
int16_t Display_height ()
```

Get the height of the initialized display. Adjusts based on the current rotation of the display.

**Returns**

int16_t height of display

#### 4.1.2.38 Display_init()

```
void Display_init (
            uint8_t options)
```

Function used to initialize the display using the given options.

**Parameters**

| | |
|---|---|
| *options* | One of the preset Adafruit INITR intialization options presets. Can be found in Adafruit_ST7735_↩ Macros.h |

#### 4.1.2.39 Display_invertDisplay()

```
void Display_invertDisplay (
            bool i)
```

Function to invert the display colors. Singleton function that automatically calls setAddrWindow.

**Parameters**

| | |
|---|---|
| *i* | True inverts the display, false sets the display to normal colors |

#### 4.1.2.40 Display_print()

```
void Display_print (
            const char * str)
```

Print a string of text to the display.

**Parameters**

| | |
|---|---|
| *str* | Pointer to the string of text to print |

#### 4.1.2.41 Display_println()

```
void Display_println (
            const char * str)
```

Print a string of text to the display followed by a newline.

**Parameters**

| | |
|---|---|
| *str* | Pointer to the string of text to print |

#### 4.1.2.42 Display_setAddrWindow()

```
void Display_setAddrWindow (
            uint16_t x,
            uint16_t y,
            uint16_t w,
            uint16_t h)
```

Sets the address windows that the display expects to receive data in from another function call. Typically not used as singleton.

**Parameters**

| | |
|---|---|
| *x* | Starting x pixel |

| | |
|---|---|
| *y* | Starting y pixel |
| *w* | Width of window |
| *h* | Height of window |

#### 4.1.2.43 Display_setCursor()

```
void Display_setCursor (
            int16_t x,
            int16_t y)
```

Sets the cursor position for the display. The cursor is used as the starting point for drawing text. Singleton function that does not call setAddrWindow.

**Parameters**

| | |
|---|---|
| *x* | x coord of the cursor |
| *y* | y coord of the cursor |

#### 4.1.2.44 Display_setFont()

```
void Display_setFont (
            const GFXfont * f)
```

Function to set the font for the display.

**Parameters**

| | |
|---|---|
| *f* | Pointer to the font to set. Fonts are defined in header files stored in Core/Inc/graphics/Fonts. |

#### 4.1.2.45 Display_setRotation()

```
void Display_setRotation (
            uint8_t m)
```

Function to change rotation of display.

**Parameters**

| | |
|---|---|
| *m* | How many 90 degree turns to apply to the display. 0 is no change, 1 rotates clockwise by 90 degrees, 2 clockwise 180 degrees, and 3 clockwise 270 degrees |

#### 4.1.2.46 Display_setTextColor()

```
void Display_setTextColor (
            uint16_t c)
```

Sets the color used to draw text on the display.

**Parameters**

| | |
|---|---|
| *c* | Color to set the text to |

#### 4.1.2.47 Display_setTextColorWithBg()

```
void Display_setTextColorWithBg (
            uint16_t c,
            uint16_t bg)
```

Sets the text color and background color behind the characters.

**Parameters**

| | |
|---|---|
| *c* | Color to draw the text in |
| *bg* | Color to draw behind the text |

#### 4.1.2.48 Display_setTextSize()

```
void Display_setTextSize (
            uint8_t sx,
            uint8_t sy)
```

Function to set the text size for the display.

**Parameters**

| | |
|---|---|
| *sx* | Horizontal size |
| *sy* | Vertical size |

#### 4.1.2.49 Display_setTextWrap()

```
void Display_setTextWrap (
            bool w)
```

Sets whether text should wrap when it reaches the edge of the display.

**Parameters**

| | |
|---|---|
| *w* | true to enable text wrapping, false to clip text at the edge of the display |

#### 4.1.2.50 Display_width()

```
int16_t Display_width ()
```

Get the width of the initialized display. Adjusts based on the current rotation of the display.

**Returns**

int16_t width of display

## 4.2 Adafruit_ST7735_API.h

Go to the documentation of this file.

```
00001
00014
00015 #ifndef ADAFRUIT_ST7735_API_H
00016 #define ADAFRUIT_ST7735_API_H
00017
00018 #include "stdint.h"
00019 #include "stm32h5xx_hal.h"
00020 #include "stdbool.h"
00021
00022 #include "Adafruit_ST7735_Macros.h"
00023 #include "Adafruit_ST77xx_Macros.h"
00024 #include "gfxfont.h"
00025 #include "stm32h5xx_hal_spi.h"
00026
00027 #ifdef __cplusplus
00028 extern "C" {
00029 #endif // __cplusplus
00030
00037 typedef struct ST7735_Handle ST7735_Handle;
00038
00039 //-------------------------------------------------------------------------------
00040 // Adafruit_ST7735.cpp function declarations
00041
00051 void Display_create(SPI_HandleTypeDef *spi_handle, int16_t cs, GPIO_TypeDef *cs_port, int16_t dc,
      GPIO_TypeDef *dc_port);
00052
00058 void Display_init(uint8_t options);
00059
00065 void Display_setRotation(uint8_t m);
00066 //-------------------------------------------------------------------------------
00067 // Adafruit_ST77xx.cpp function declarations
00068
00077 void Display_setAddrWindow(uint16_t x, uint16_t y, uint16_t w, uint16_t h);
00078
00084 void Display_enableDisplay(bool enable);
00085
00091 void Display_enableTearing(bool enable);
00092
00098 void Display_enableSleep(bool enable);
00099
00100 //-------------------------------------------------------------------------------
00101 // Adafruit_SPITFT.cpp function declarations
00102
00110 void Display_drawPixel(int16_t x, int16_t y, uint16_t color);
00111
00121 void Display_fillRect(int16_t x, int16_t y, int16_t w, int16_t h, uint16_t color);
00122
00131 void Display_drawFastHLine(int16_t x, int16_t y, int16_t w, uint16_t color);
00132
00141 void Display_drawFastVLine(int16_t x, int16_t y, int16_t h, uint16_t color);
00142
00148 void Display_invertDisplay(bool i);
00149
```

```
uint16_t Display_color565(uint8_t r, uint8_t g, uint8_t b);

void Display_drawRGBBitmap(int16_t x, int16_t y, uint16_t *pcolors, int16_t w, int16_t h);

void Display_drawRGBBitmapMask(int16_t x, int16_t y, uint16_t *pcolors, int16_t w, int16_t h, uint8_t *mask);

//-----------------------------------------------------------------------------
// Adafruit_GFX.cpp function declarations

void Display_fillScreen(uint16_t color);

void Display_drawLine(int16_t x0, int16_t y0, int16_t x1, int16_t y1, uint16_t color);

void Display_drawRect(int16_t x, int16_t y, int16_t w, int16_t h, uint16_t color);

void Display_drawCircle(int16_t x0, int16_t y0, int16_t r, uint16_t color);

void Display_drawCircleHelper(int16_t x0, int16_t y0, int16_t r, uint8_t cornername, uint16_t color);

void Display_fillCircle(int16_t x0, int16_t y0, int16_t r, uint16_t color);

void Display_fillCircleHelper(int16_t x0, int16_t y0, int16_t r, uint8_t cornername, int16_t delta, uint16_t color);

void Display_drawEllipse(int16_t x0, int16_t y0, int16_t rw, int16_t rh, uint16_t color);

void Display_fillEllipse(int16_t x0, int16_t y0, int16_t rw, int16_t rh, uint16_t color);

void Display_drawTriangle(int16_t x0, int16_t y0, int16_t x1, int16_t y1, int16_t x2, int16_t y2, uint16_t color);

void Display_fillTriangle(int16_t x0, int16_t y0, int16_t x1, int16_t y1, int16_t x2, int16_t y2, uint16_t color);

void Display_drawRoundRect(int16_t x0, int16_t y0, int16_t w, int16_t h, int16_t radius, uint16_t color);

void Display_fillRoundRect(int16_t x0, int16_t y0, int16_t w, int16_t h, int16_t radius, uint16_t color);

void Display_drawBitmap(int16_t x, int16_t y, uint8_t *bitmap, int16_t w, int16_t h, uint16_t color);

void Display_drawBitmapBg(int16_t x, int16_t y, uint8_t *bitmap, int16_t w, int16_t h, uint16_t color, uint16_t bg);

void Display_drawXBitmap(int16_t x, int16_t y, uint8_t *bitmap, int16_t w, int16_t h, uint16_t color);

void Display_drawXBitmapBg(int16_t x, int16_t y, uint8_t *bitmap, int16_t w, int16_t h, uint16_t color, uint16_t bg);

void Display_drawGrayscaleBitmap(int16_t x, int16_t y, uint8_t *bitmap, int16_t w, int16_t h);

void Display_drawGrayscaleBitmapMask(int16_t x, int16_t y, uint8_t *bitmap, int16_t w, int16_t h, uint8_t *mask);

void Display_drawChar(int16_t x, int16_t y, unsigned char c, uint16_t color, uint16_t bg, uint8_t size_x, uint8_t size_y);

void Display_getTextBounds(const char *string, int16_t x, int16_t y, int16_t *x1, int16_t *y1, uint16_t *w, uint16_t *h);

void Display_setTextSize(uint8_t sx, uint8_t sy);

void Display_setFont(const GFXfont *f);

void Display_setCursor(int16_t x, int16_t y);

void Display_setTextColor(uint16_t c);

void Display_setTextColorWithBg(uint16_t c, uint16_t bg);

void Display_setTextWrap(bool w);

void Display_cp437(bool x);

int16_t Display_width();

int16_t Display_height();

uint8_t Display_getRotation();

int16_t Display_getCursorX();

int16_t Display_getCursorY();

void Display_print(const char *str);
```

```
00521
00527 void Display_println(const char *str);
00528
00529 #ifdef __cplusplus
00530 }
00531 #endif // __cplusplus
00532
00533 #endif // ADAFRUIT_ST7735_API_H
```

## 4.3 Core/Inc/graphics/Adafruit_TFTShield18_API.h File Reference

```
#include "stdint.h"
#include "Adafruit_TFTShield18_Macros.h"
#include "stm32h5xx_hal.h"
```

### Typedefs

- typedef struct TFTShield18_Handle TFTShield18_Handle

  *Internal handle used to store the instance of an Adafruit_TFTShield18 class for use in the C API functions. This struct is not intended to be used outside of this API and should be treated as an opaque handle. Provides a C compatible structure to store the C++ class instance for the TFT shield.*

### Functions

- void TFTShield18_create (I2C_HandleTypeDef *i2c_handle)

  *Function to create an instance of the seesaw class for the TFT shield. Must be called before any other functions in this API.*
- bool TFTShield18_begin ()

  *Function to initialize the seesaw device on the TFT shield. Must be called before using any other functions in this API. Uses the default I2C address of 0x3E for the TFT seesaw device.*
- void TFTShield18_setBacklight (uint16_t value)

  *Sets the backlight to a value between 0(0x0000) (off) and 65535(0xFFFF) (full brightness).*
- void TFTShield18_setBacklightFreq (uint16_t freq)

  *Sets the frequency of the backlight PWM.*
- void TFTShield18_tftReset (bool rst)

  *Perform a SW reset of the display and seesaw devices. Required to be called after initialization to reset the display from an unknown state. Should be called whenever a reset of the display is needed by setting rst to true and then back to false after a delay.*
- uint32_t TFTShield18_readButtons ()

  *Function to read the state of the on board shield buttons from the seesaw chip over I2C. Returned bitmask has bits set corresponding to the TFTSHIELD_BUTTON macros defined in Adafruit_TFTShield18_Macros.h.*

### 4.3.1 Typedef Documentation

#### 4.3.1.1 TFTShield18_Handle

```
typedef struct TFTShield18_Handle TFTShield18_Handle
```

Internal handle used to store the instance of an Adafruit_TFTShield18 class for use in the C API functions. This struct is not intended to be used outside of this API and should be treated as an opaque handle. Provides a C compatible structure to store the C++ class instance for the TFT shield.

### 4.3.2 Function Documentation

#### 4.3.2.1 TFTShield18_begin()

```
bool TFTShield18_begin ()
```

Function to initialize the seesaw device on the TFT shield. Must be called before using any other functions in this API. Uses the default I2C address of 0x3E for the TFT seesaw device.

**Returns**

true if the initialization was successful

false if the initialization failed

#### 4.3.2.2 TFTShield18_create()

```
void TFTShield18_create (
            I2C_HandleTypeDef * i2c_handle)
```

Function to create an instance of the seesaw class for the TFT shield. Must be called before any other functions in this API.

**Parameters**

| | |
|---|---|
| *i2c_handle* | Pointer to the STM32 HAL I2C handle to use for communication with the TFT seesaw device |

#### 4.3.2.3 TFTShield18_readButtons()

```
uint32_t TFTShield18_readButtons ()
```

Function to read the state of the on board shield buttons from the seesaw chip over I2C. Returned bitmask has bits set corresponding to the TFTSHIELD_BUTTON macros defined in Adafruit_TFTShield18_Macros.h.

**Returns**

uint32_t Bitmask representing the state of the buttons

#### 4.3.2.4 TFTShield18_setBacklight()

```
void TFTShield18_setBacklight (
            uint16_t value)
```

Sets the backlight to a value between 0(0x0000) (off) and 65535(0xFFFF) (full brightness).

**Parameters**

| | |
|---|---|
| *value* | Value to set the backlight to (between 0 and 65535) |

#### 4.3.2.5 TFTShield18_setBacklightFreq()

```
void TFTShield18_setBacklightFreq (
            uint16_t freq)
```

Sets the frequency of the backlight PWM.

**Parameters**

| | |
|---|---|
| *freq* | Frequency to set the backlight PWM to |

#### 4.3.2.6 TFTShield18_tftReset()

```
void TFTShield18_tftReset (
            bool rst)
```

Perform a SW reset of the display and seesaw devices. Required to be called after initialization to reset the display from an unknown state. Should be called whenever a reset of the display is needed by setting rst to true and then back to false after a delay.

**Parameters**

| | |
|---|---|
| *rst* | Boolean value to trigger the reset. Set to true to initiate the reset and false to complete it. |

## 4.4 Adafruit_TFTShield18_API.h

Go to the documentation of this file.

```
00001
00013
00014 #ifndef ADAFRUIT_TFTSHIELD18_API_H
00015 #define ADAFRUIT_TFTSHIELD18_API_H
00016
00017 #include "stdint.h"
00018 #include "Adafruit_TFTShield18_Macros.h"
00019 #include "stm32h5xx_hal.h"
00020
00021 #ifdef __cplusplus
00022 extern "C" {
00023 #endif // __cplusplus
00024
00031 typedef struct TFTShield18_Handle TFTShield18_Handle;
00032
00038 void TFTShield18_create(I2C_HandleTypeDef *i2c_handle);
00039
00047 bool TFTShield18_begin();
00048
00054 void TFTShield18_setBacklight(uint16_t value);
00055
00061 void TFTShield18_setBacklightFreq(uint16_t freq);
00062
00069 void TFTShield18_tftReset(bool rst);
00070
00077 uint32_t TFTShield18_readButtons();
00078
00079 #ifdef __cplusplus
00080 }
00081 #endif // __cplusplus
00082
00083 #endif // ADAFRUIT_TFTSHIELD18_API_H
```

# Index


Adafruit_ST7735_API.h
- Display_color565, 10
- Display_cp437, 10
- Display_create, 10
- Display_drawBitmap, 11
- Display_drawBitmapBg, 11
- Display_drawChar, 12
- Display_drawCircle, 12
- Display_drawCircleHelper, 13
- Display_drawEllipse, 13
- Display_drawFastHLine, 14
- Display_drawFastVLine, 14
- Display_drawGrayscaleBitmap, 14
- Display_drawGrayscaleBitmapMask, 15
- Display_drawLine, 15
- Display_drawPixel, 16
- Display_drawRect, 16
- Display_drawRGBBitmap, 16
- Display_drawRGBBitmapMask, 17
- Display_drawRoundRect, 17
- Display_drawTriangle, 18
- Display_drawXBitmap, 18
- Display_drawXBitmapBg, 19
- Display_enableDisplay, 19
- Display_enableSleep, 19
- Display_enableTearing, 20
- Display_fillCircle, 20
- Display_fillCircleHelper, 20
- Display_fillEllipse, 21
- Display_fillRect, 21
- Display_fillRoundRect, 22
- Display_fillScreen, 22
- Display_fillTriangle, 22
- Display_getCursorX, 23
- Display_getCursorY, 23
- Display_getRotation, 23
- Display_getTextBounds, 23
- Display_height, 24
- Display_init, 24
- Display_invertDisplay, 24
- Display_print, 25
- Display_println, 25
- Display_setAddrWindow, 25
- Display_setCursor, 26
- Display_setFont, 26
- Display_setRotation, 26
- Display_setTextColor, 26
- Display_setTextColorWithBg, 27
- Display_setTextSize, 27
- Display_setTextWrap, 27
- Display_width, 28
- ST7735_Handle, 10

Adafruit_TFTShield18_API.h
- TFTShield18_begin, 31
- TFTShield18_create, 31
- TFTShield18_Handle, 30
- TFTShield18_readButtons, 31
- TFTShield18_setBacklight, 31
- TFTShield18_setBacklightFreq, 31
- TFTShield18_tftReset, 32

Core Directory Reference, 5
Core/Inc Directory Reference, 5
Core/Inc/graphics Directory Reference, 5
Core/Inc/graphics/Adafruit_ST7735_API.h, 7, 28
Core/Inc/graphics/Adafruit_TFTShield18_API.h, 30, 32

Display_color565
- Adafruit_ST7735_API.h, 10

Display_cp437
- Adafruit_ST7735_API.h, 10

Display_create
- Adafruit_ST7735_API.h, 10

Display_drawBitmap
- Adafruit_ST7735_API.h, 11

Display_drawBitmapBg
- Adafruit_ST7735_API.h, 11

Display_drawChar
- Adafruit_ST7735_API.h, 12

Display_drawCircle
- Adafruit_ST7735_API.h, 12

Display_drawCircleHelper
- Adafruit_ST7735_API.h, 13

Display_drawEllipse
- Adafruit_ST7735_API.h, 13

Display_drawFastHLine
- Adafruit_ST7735_API.h, 14

Display_drawFastVLine
- Adafruit_ST7735_API.h, 14

Display_drawGrayscaleBitmap
- Adafruit_ST7735_API.h, 14

Display_drawGrayscaleBitmapMask
- Adafruit_ST7735_API.h, 15

Display_drawLine
- Adafruit_ST7735_API.h, 15

Display_drawPixel
- Adafruit_ST7735_API.h, 16

Display_drawRect
- Adafruit_ST7735_API.h, 16

Display_drawRGBBitmap
Adafruit_ST7735_API.h, 16
Display_drawRGBBitmapMask
Adafruit_ST7735_API.h, 17
Display_drawRoundRect
Adafruit_ST7735_API.h, 17
Display_drawTriangle
Adafruit_ST7735_API.h, 18
Display_drawXBitmap
Adafruit_ST7735_API.h, 18
Display_drawXBitmapBg
Adafruit_ST7735_API.h, 19
Display_enableDisplay
Adafruit_ST7735_API.h, 19
Display_enableSleep
Adafruit_ST7735_API.h, 19
Display_enableTearing
Adafruit_ST7735_API.h, 20
Display_fillCircle
Adafruit_ST7735_API.h, 20
Display_fillCircleHelper
Adafruit_ST7735_API.h, 20
Display_fillEllipse
Adafruit_ST7735_API.h, 21
Display_fillRect
Adafruit_ST7735_API.h, 21
Display_fillRoundRect
Adafruit_ST7735_API.h, 22
Display_fillScreen
Adafruit_ST7735_API.h, 22
Display_fillTriangle
Adafruit_ST7735_API.h, 22
Display_getCursorX
Adafruit_ST7735_API.h, 23
Display_getCursorY
Adafruit_ST7735_API.h, 23
Display_getRotation
Adafruit_ST7735_API.h, 23
Display_getTextBounds
Adafruit_ST7735_API.h, 23
Display_height
Adafruit_ST7735_API.h, 24
Display_init
Adafruit_ST7735_API.h, 24
Display_invertDisplay
Adafruit_ST7735_API.h, 24
Display_print
Adafruit_ST7735_API.h, 25
Display_println
Adafruit_ST7735_API.h, 25
Display_setAddrWindow
Adafruit_ST7735_API.h, 25
Display_setCursor
Adafruit_ST7735_API.h, 26
Display_setFont
Adafruit_ST7735_API.h, 26
Display_setRotation
Adafruit_ST7735_API.h, 26
Display_setTextColor
Adafruit_ST7735_API.h, 26
Display_setTextColorWithBg
Adafruit_ST7735_API.h, 27
Display_setTextSize
Adafruit_ST7735_API.h, 27
Display_setTextWrap
Adafruit_ST7735_API.h, 27
Display_width
Adafruit_ST7735_API.h, 28

ST7735_Handle
Adafruit_ST7735_API.h, 10

TFTShield18_begin
Adafruit_TFTShield18_API.h, 31
TFTShield18_create
Adafruit_TFTShield18_API.h, 31
TFTShield18_Handle
Adafruit_TFTShield18_API.h, 30
TFTShield18_readButtons
Adafruit_TFTShield18_API.h, 31
TFTShield18_setBacklight
Adafruit_TFTShield18_API.h, 31
TFTShield18_setBacklightFreq
Adafruit_TFTShield18_API.h, 31
TFTShield18_tftReset
Adafruit_TFTShield18_API.h, 32